CSCI 264 — Assembly Language

Lab 6

Writing Assembly in the ASM Visualizer

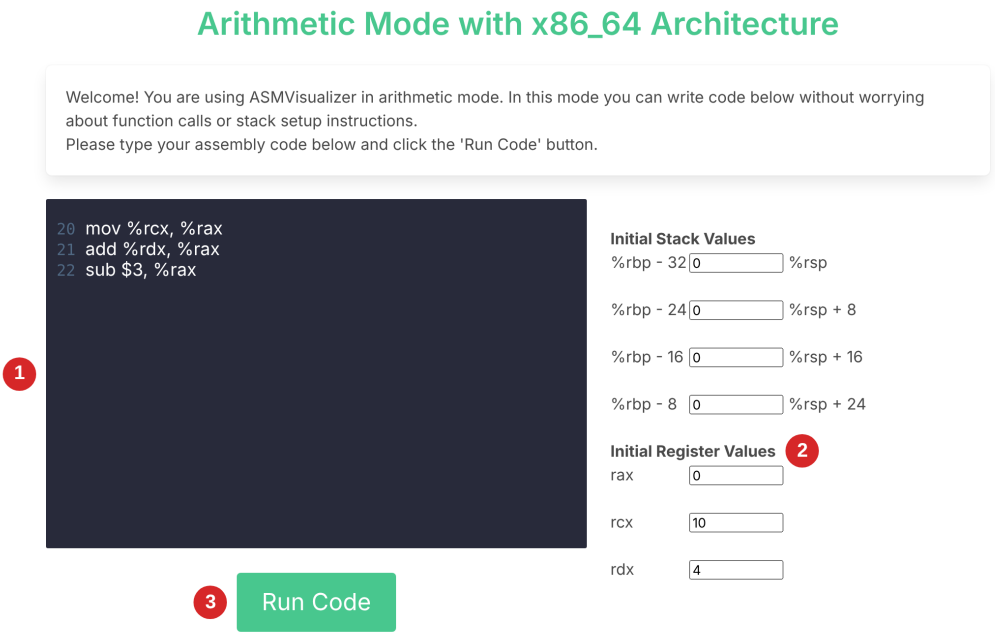
In this lab you write your first assembly programs, using `mov`, `add`, and `sub`. You will write and run them in the **ASM Visualizer**, which runs your code one instruction at a time and shows you the registers and the stack after each step. Work through the example in the first section before you start the problems.

The ASM Visualizer

Open this page in a browser. You do not need an account:

`https://asm.diveintosystems.org/arithmetic/x86_64`

The title at the top should say **Arithmetic Mode with x86_64 Architecture**. If it says x86 or arm64, you are in the wrong version: use the link above.



1 **The code box.** Type one instruction per line. Anything after a `#` is a comment. The line numbers start at 20, not 1: the visualizer adds 19 lines of setup code before yours and hides them.

2 **Starting values.** Set the starting value of `%rax`, `%rcx`, and `%rdx` here. The boxes above them set four 8-byte slots on the stack, which you only need for Problem 5. Type

numbers in decimal (`10`, `-3`) or in hex starting with `0x` (`0x1F`). Anything you don't set starts at 0.

3 **Run Code** runs your program and takes you to the results page.

Example

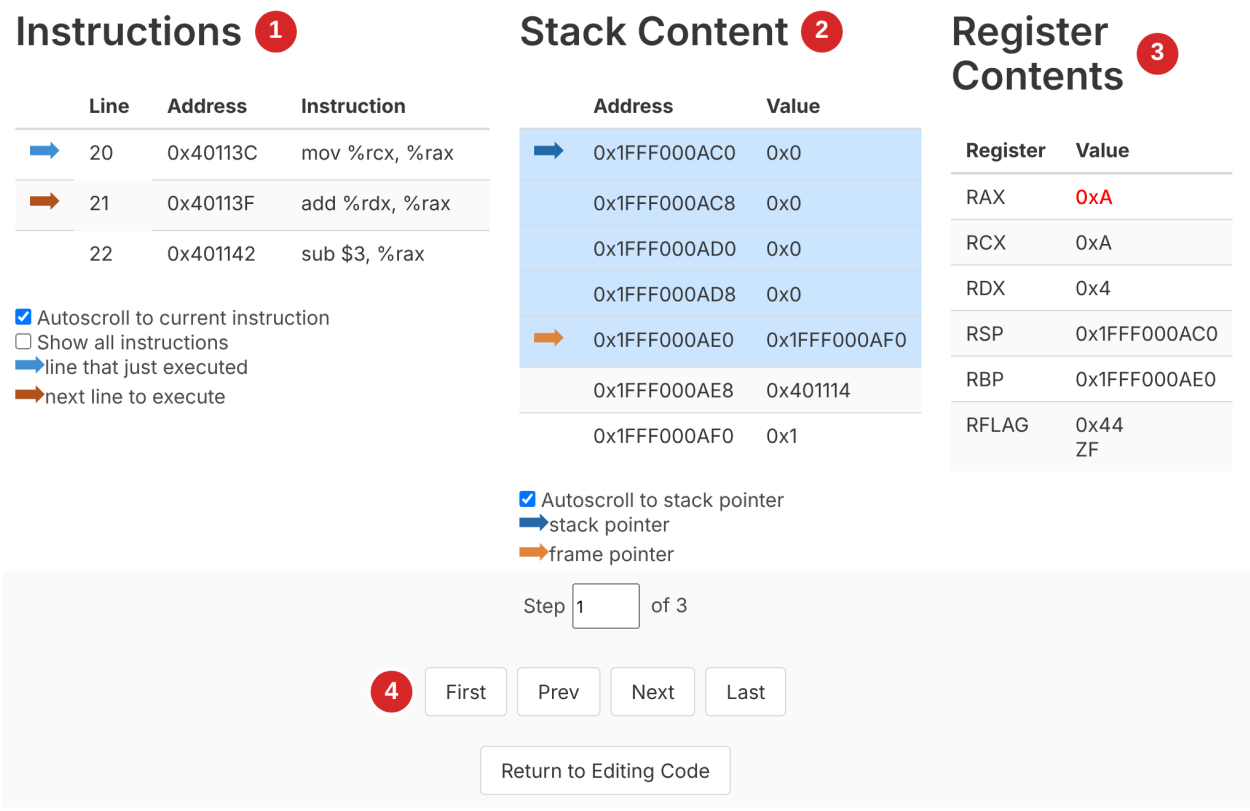
Suppose `x` is in `%rcx` and `y` is in `%rdx`, and we want `x + y - 3` in `%rax`:

```
mov %rcx, %rax      # rax = x
add %rdx, %rax      # rax = x + y
sub $3, %rax        # rax = x + y - 3
```

Type it into the code box, set `rcx` to 10 and `rdx` to 4, and click **Run Code**. (The comments are left out of the pictures.)

Stepping Through Your Code

The results page has three tables. This is the example after one step:



1 **Instructions.** The blue arrow is the line that just ran. The orange arrow is the line that runs next.

2 **Stack Content.** Each row is 8 bytes of the stack, with the lowest address at the top. The blue arrow is `%rsp` and the orange arrow is `%rbp`. You only need this for Problem 5.

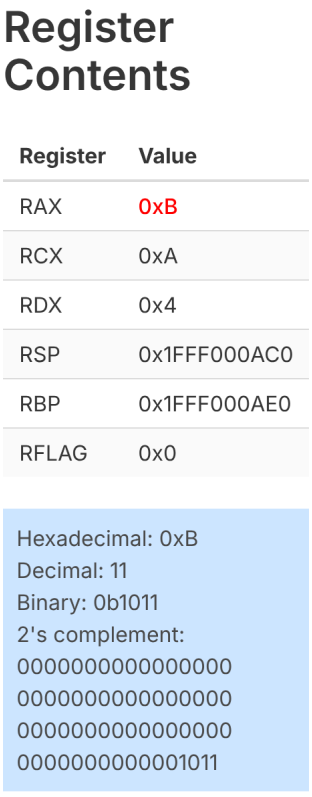
3 **Register Contents.** Only the registers your code uses are listed, plus `RSP`, `RBP`, and `RFLAG`. Ignore `RFLAG` for now. **A value in red just changed.** Here `mov` has just copied `%rcx` into `%rax`, so `RAX` is now `0xA`.

4 **Step controls.** Step 0 is before any of your code has run. **Next** and **Prev** move one instruction at a time, and **First** and **Last** jump to the start and the end. You can also type a step number in the box. **Return to Editing Code** takes you back to your code, with your starting values still filled in.

Values are shown in hex. Hover over a value to see it in decimal, binary, and two's complement. The picture on the right is the last step of the example, with the mouse over `RAX`: it is `0xB`, which is 11.
Negative numbers are shown in two's complement. For example, -7 is shown as:

`0xFFFFFFFFFFFFFFF9`

Hover over it and it says `Decimal: -7`.



Saving Your Work

The visualizer does not save your code. Keep your code in a text file and copy it into the visualizer.

At the bottom of the editing page is a button, **Click here to share the link of the current code!** It copies a link that has your code and starting values built in, and shows the link in a pop-up. Opening that link later brings back your code and values exactly as they were.

Lab 6 Problems

For each problem, I give you some C-like psuedocode. Rewrite it in Assembly using the ASM Visualizer. In every problem:

- Start with `x` in `%rcx` and `y` in `%rdx`. Set them in the starting value boxes.
- Put the result in `%rax`.
- Use only `mov`, `add`, and `sub`. If you need more registers, use `%r8` through `%r11`.

Run each problem with both tests. The first test gives the answer you should get. For the second, work out the answer by hand *before* you run it, then check it.

Keep your work in a text file called `lab6.txt`, set up like this:

```
Name:
Helped by:
Generative AI use:

Problem 1
(your assembly, with comments)
Link:
Test 2: I expected ____, the visualizer gave ____

Problem 2
...
```

For each *Link*, paste the share link for that problem.

Problem 1

```
result = y - x;
```

Test 1: `x` = 3, `y` = 10. Then `%rax` is 7.
Test 2: `x` = 10, `y` = 3.

Problem 2

Swap `x` and `y`:

```
temp = x;
x = y;
y = temp;
```

There is no result in `%rax` for this problem.
Test 1: `x` = 3, `y` = 8. Then `%rcx` is 8 and `%rdx` is 3.
Test 2: `x` = −1, `y` = 5.

Problem 3

```
x = x + y;
y = x - y;
x = x - y;
```

Remember that `sub S, D` computes `D - S`. Run both tests, then answer in a comment: what does this code do to `x` and `y`? There is no result in `%rax` for this problem.
Test 1: `x` = 3, `y` = 8.
Test 2: `x` = −4, `y` = 6.

Problem 4

```
result = 2*x + 3*y - 10;
```

We have not covered a multiply instruction yet. You can look up on your own how to do it, or do it with `add`.
Test 1: `x` = 4, `y` = 5. Then `%rax` is 13.
Test 2: `x` = −1, `y` = 2.

Problem 5

This time the variables are on the stack, like the compiler's code from lecture: `a` is at `-8(%rbp)` and `b` is at `-16(%rbp)`. Set them in the **Initial Stack Values** boxes.

```
a = a + b;
result = a;
```

Store the new `a` back at `-8(%rbp)`. An instruction can't have memory as both its source and its destination. In the Stack Content table, `-8(%rbp)` is the row just above the orange arrow.
Test 1: `a` = 20, `b` = 22. Then `a` and `%rax` are 42 (`0x2A`).
Test 2: `a` = 7, `b` = −10.

Submission

Submit `lab6.txt` on Canvas by Wednesday, October 21 and demo to Professor West. There is no lab on October 20 (Fall Break). Finish the example and Problems 1 and 2 before you leave lab today. If a classmate helped you, put their name in the file. Disclose any use of Generative AI at the top.